

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение

высшего образования

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет программной инженерии и компьютерной техники

Образовательная программа: Системное и прикладное программное
обеспечение

Направление подготовки: Программная инженерия 09.04.04

Дисциплина: Системное программное обеспечение

Практическое задание №1

Обучающийся: Пименов Глеб Андреевич 506758

Группа: Р4115

Дата 20.01.2026

Санкт-Петербург

2026

Оглавление

1. Цели	2
2. Задачи	3
3. Описание работы	6
4. Аспекты реализации	6
4.1. Средство синтаксического анализа	6
4.2. Структура программы	7
4.3. Арифметические операторы:	7
4.4. Логические операторы:	8
4.5. Операторы сравнения:	8
4.6. Операторы инкремента/декремента:	8
4.7. Операторы присваивания:	8
4.8. Управляющие конструкции	8
4.9. Состав и назначение модулей	9
4.10. Примеры использования	11
Результаты	13
Выводы	14

1. Цели

Использовать средство синтаксического анализа по выбору, реализовать модуль для разбора текста в соответствии с языком по варианту. Реализовать построение по исходному файлу с текстом синтаксического дерева с узлами, соответствующими элементам синтаксической модели языка. Вывести полученное дерево в

файл в формате, поддерживающем просмотр графического представления.

2. Задачи

1. Изучить выбранное средство синтаксического анализа

a. Средство должно поддерживать программный интерфейс, совместимый с языком Си

b. Средство должно параметризоваться спецификацией, описывающей синтаксическую структуру разбираемого языка

c. Средство может функционировать посредством кодогенерации и/или подключения

необходимых для его работы дополнительных библиотек

d. Средство может быть реализовано с нуля, в этом случае оно должно использовать

обобщённый алгоритм, управляемый спецификацией

2. Изучить синтаксис разбираемого по варианту языка и записать спецификацию для средства

синтаксического анализа, включающую следующие конструкции:

a. Подпрограммы со списком аргументов и возвращаемым значением

b. Операции контроля потока управления – простые ветвления if-else и циклы или аналоги

c. В зависимости от варианта – определения переменных

d. Целочисленные, строковые и односимвольные литералы

e. Выражения численной, битовой и логической арифметики

f. Выражения над одномерными массивами

g. Выражения вызова функции

3. Реализовать модуль, использующий средство синтаксического анализа для разбора языка по варианту

- a. Программный интерфейс модуля должен принимать строку с текстом и возвращать структуру, описывающую соответствующее дерево разбора и коллекцию сообщений об ошибке
 - b. Результат работы модуля – дерево разбора – должно содержать иерархическое представление для всех синтаксических конструкций, включая выражения, логически представляющие собой иерархически организованные данные, даже если на уровне средства синтаксического анализа для их разбора было использовано линейное представление
4. Реализовать тестовую программу для демонстрации работоспособности созданного модуля
- a. Через аргументы командной строки программа должна принимать имя входного файла для чтения и анализа, имя выходного файла записи для дерева, описывающего синтаксическую структуру разобранного текста
 - b. Сообщения об ошибке должны выводиться тестовой программой (не модулем, отвечающим за анализ!) в стандартный поток вывода ошибок
5. Результаты тестирования представить в виде отчета, в который включить:
- a. В части 3 привести описание структур данных, представляющих результат разбора текста (3а)
 - b. В части 4 описать, какая дополнительная обработка потребовалась для результата разбора,

предоставляемого средством синтаксического анализа, чтобы
сформировать результат работы
созданного модуля
с. В части 5 привести примеры исходных анализируемых текстов для всех
синтаксических
конструкций разбираемого языка и соответствующие результаты разбора

Вариант 3

```
source: sourceItem*;
typeRef: {
|builtin: 'bool'|'byte'|'int'|'uint'|'long'|'ulong'|'char'|'string';
|custom: identifier;
|array: typeRef '(' (',')* ')';
};
funcSignature: identifier '(' list<argDef> ')' ('as' typeRef)? {
argDef: identifier ('as' typeRef)?;
};
sourceItem: {
|funcDef: 'function' funcSignature statement* 'end' 'function';
};
statement: {
|var: 'dim' list<identifier> 'as' typeRef; // for static typing
|if: 'if' expr 'then' statement* ('else' statement*)? 'end' 'if';
|while: 'while' expr statement* 'wend';
|do: 'do' statement* 'loop' ('while'|'until') expr;
|break: 'break';
|expression: expr ';';
};
expr: { // присваивание через '='
```

```
|binary: expr binOp expr; // где binOp - символ бинарного оператора
|unary: unOp expr; // где unOp - символ унарного оператора
|braces: '(' expr ')';
|callOrIndexer: expr '(' list<expr> ')';
|place: identifier;
|literal: bool|str|char|hex|bits|dec;
};
```

3. Описание работы

Разработанная программа реализует систему для синтаксического анализа специализированного языка программирования с функциональным парадигмой. Основное назначение - преобразование текстового исходного кода в структурированное синтаксическое дерево (Abstract Syntax Tree, AST), которое содержит иерархическое представление всех синтаксических конструкций программы. Полученное дерево сохраняется в формате Mermaid Flowchart, что позволяет визуализировать структуру анализируемого кода.

4. Аспекты реализации

4.1. Средство синтаксического анализа

В качестве средства синтаксического анализа выбраны инструменты Flex (лексический анализатор) и Bison (синтаксический анализатор). Эти инструменты соответствуют всем требованиям задания:

- Совместимость с Си: Оба инструмента генерируют код на C, полностью интегрируемый с остальной программой
- Параметризация спецификацией: Грамматика языка описывается в файле `parser.y`, а таблица токенов в `lexems.l`

- Кодогенерация: Bison генерирует парсер на C (parser.tab.c, parser.tab.h), Flex генерирует лексер (lex.yy.c)
- Интеграция: Сгенерированные модули подключаются в основную программу через стандартный интерфейс

4.2. Структура программы

Программа состоит из набора функций, каждая из которых имеет:

- Сигнатуру с именем, списком параметров и типом возвращаемого значения
- Тело функции со списком операторов

```
function fibonacci(n as int) as int
    if n <= 1 then
        n;
    else
        fibonacci(n - 1) + fibonacci(n - 2);
    end if
end function
```

Язык поддерживает следующие встроенные типы:

- int - целые числа
- byte - однобайтовые значения
- uint - беззнаковые целые числа
- Массивы встроенных типов с произвольным числом измерений

А также 'bool', 'long', 'ulong', 'char', 'string';

4.3. Арифметические операторы:

- + - сложение
- - - вычитание
- * - умножение

- / - целочисленное деление
- % - остаток от деления

4.4. Логические операторы:

- and - логическое И
- or - логическое ИЛИ
- not - логическое НЕ

4.5. Операторы сравнения:

- == - равенство
- != - неравенство
- <, > - меньше, больше
- <=, >= - меньше или равно, больше или равно

4.6. Операторы инкремента/декремента:

- ++ - инкремент
- -- - декремент

4.7. Операторы присваивания:

- = - присваивание значения

4.8. Управляющие конструкции

Условный оператор IF-ELSE:

if условие then

 операторы_если_верно

else

 операторы_если_ложь

end if

Циклы WHILE и UNTIL:

while условие

операторы

wend

do

операторы

wend until условие

Оператор BREAK:

Прерывает текущий цикл и переходит к следующему оператору.

Доступ к элементам массива осуществляется через синтаксис индексации:

array[index1][index2]

Объявление массива: dim myArray as int(,)

Функции вызываются с передачей аргументов: result = fibonacci(n - 1) +
fibonacci(n - 2);

4.9. Состав и назначение модулей

Модуль лексического анализа ('lexems.l')

Назначение: Разбор входного текста на лексемы (токены).

Функциональность:

- Распознавание ключевых слов ('function', 'if', 'while', 'dim' и т.д.)
- Распознавание идентификаторов и переменных
- Преобразование литералов в машинное представление
- Пропуск комментариев и пробелов
- Отслеживание номеров строк для сообщений об ошибках

Выходные данные: Поток токенов с типами из 'parser.tab.h'.

Модуль синтаксического анализа ('parser.y')

Назначение: Построение абстрактного синтаксического дерева на основе потока токенов.

Функциональность:

- Проверка синтаксической корректности программы
- Построение иерархического представления структурных элементов
- Управление приоритетами операторов

Модуль представления дерева (`node.c/h`)

Структуры данных:

```
struct TreeNode {  
    char *type;          // Тип узла ("funcDef", "if", "expr", и т.д.)  
    TreeNode **childNodes; // Массив дочерних узлов  
    int childrenNumber;   // Количество дочерних узлов  
    char *value;          // Значение узла (для литералов и идентификаторов)  
    int id;               // Уникальный идентификатор узла  
};  
  
struct ChildNodes {  
    int size;             // Количество элементов  
    TreeNode **childNodes; // Массив дочерних узлов  
};
```

Модуль парсера (`parser.c/h`)

Функции:

```
ParseResult* parse(FILE *file);  
void freeMem(ParseResult *parseResult);
```

Структура результата:

```
struct ParseResult {  
    int size;             // Количество узлов в дереве  
    TreeNode **nodes;     // Массив всех узлов  
    char** errors;        // Массив сообщений об ошибках  
    int errorsCount;      // Количество ошибок  
};
```

4.10. Примеры использования

Работа проверялась на ubuntu 20.04, а также на wsl

Простой пример

Входной файл (`calc.txt`):

```
function add(a as int, b as int) as int
    a + b;
end function
```

```
function main() as int
```

```
    dim x as int
```

```
    dim y as int
```

```
    x = 5;
```

```
    y = 3;
```

```
    add(x, y);
```

```
end function
```

Команда запуска:

```
cd build
```

```
make .
```

```
./SYSSOFT ../output ../input/calc.txt
```

Выходные файлы:

- output/calc.txt-parse-tree.txt - AST в формате Mermaid

Сложный пример с циклами и условиями

Входной файл (fib.txt):

```
function fibonacci(n as int) as int
    if n <= 1 then
```

```
    n;  
else  
    fibonacci(n - 1) + fibonacci(n - 2);  
end if  
end function
```

```
function read_num() as int  
    dim in as int  
    dim res as int  
    while true  
        in = input();  
        if in == 13 then  
            break  
        end if  
        res = res * 10 + (in - 48);  
    wend  
    res;  
end function
```

Формат выходного файла (AST)

Пример выходного Mermaid Flowchart:

flowchart TB

node0([Type: DEC, Value: 42])

node1([Type: IDENTIFIER, Value: x])

node2([Type: expr]) --> node1

node3([Type: binary]) --> node2

Результаты

В результате работы были созданы: лексический анализатор (source/lexems.l), синтаксический анализатор (source/parser.y), генерация узлов AST в процессе разбора, модуль представления дерева (node.c/h), структуры TreeNode, ChildNodes, ParseResult, модуль обработки парсинга (parser.c/h), сборка результата в структуру ParseResult.

Пример теста для текста:

```
function read_num() as int
    dim in as int
    dim res as int
    res = 0;
    while true
        in = input();
        if in == 13 then
            break
        end if
        res = res * 10 + (in - 48);
    wend
    res;
end function
```

Синтаксическое дерево:

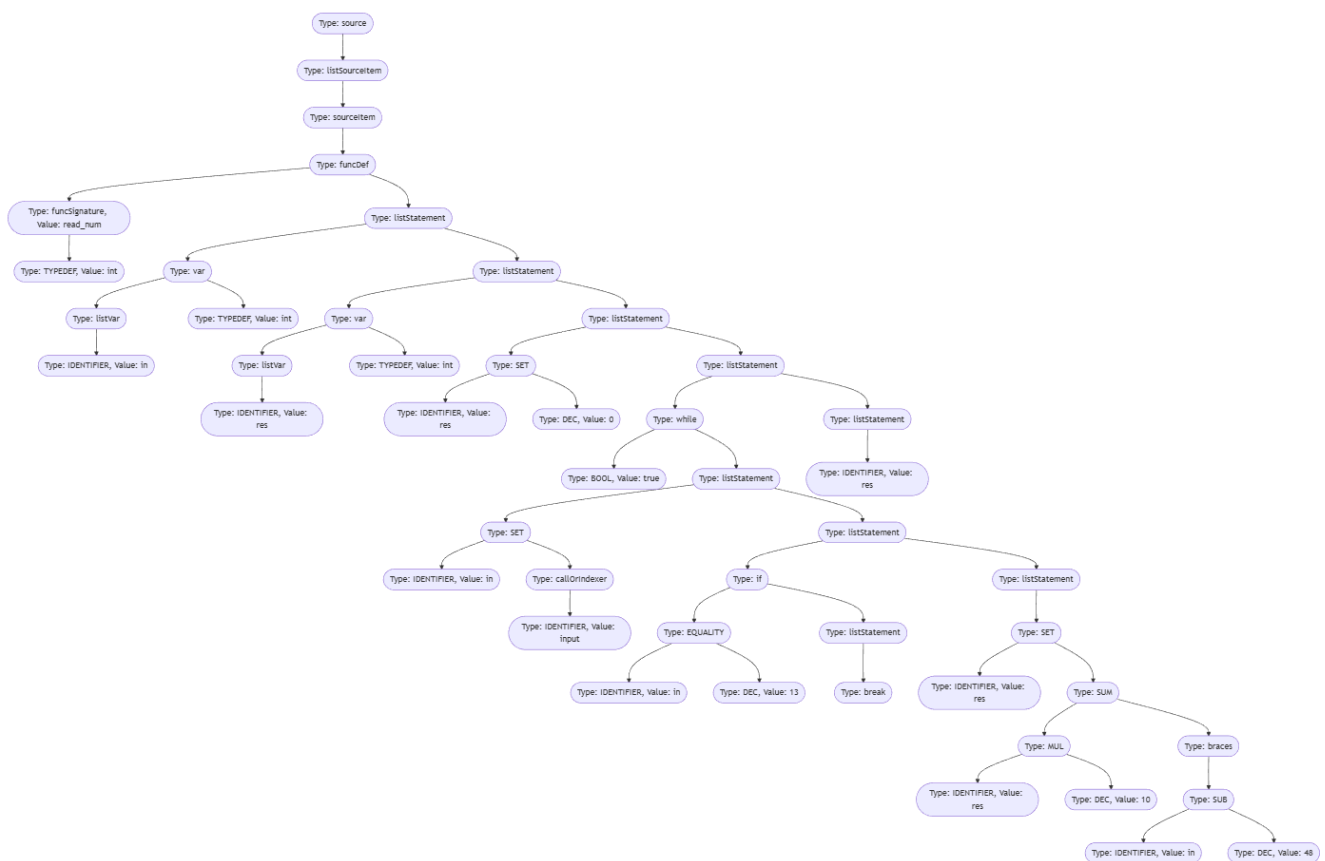


Рисунок 1 - синтаксическое дерево

Выводы

Лабораторная работа успешно позволила глубоко понять принципы синтаксического анализа. Созданная система является полностью функциональной и готовой к использованию, соответствует всем требованиям задания, протестирована на реальных примерах программ. Может быть использована как основа для следующих лабораторных. Практическое применение инструментов Flex и Bison подтвердило их эффективность и стандартизированность в разработке компиляторов. Опыт разработки показал, что правильный выбор инструментов и архитектуры позволяет сосредоточиться на логике приложения, а не на низкоуровневых деталях реализации парсера.